

NDAHIRO Putine Jonathan

Matriculation num: 4242218

PROJECT Cloud Programming (DLBSEPCP01_E)

PHASE 2: DEVELOPMENT PHASE

GLOBAL HIGH-AVAILABILITY Website on AWS with CloudFront | multi-AZ | Auto-Scaling

GitHub: <https://github.com/CODEWITHNDAHIRO/DLBSEPCP01-cloud-programming-AWS>

COMPLETE 4 LEVER ARCHITECTURE (AS IN CONCEPTION PHASE)

Layer 1: CloudFront

600+ edge locations

Global distribution Low

Latency cache

layer 2: Application Load Balancer

Public subnets 2 AZs

Health checks

Layer 3: Auto Scaling Group (ASG)

Private subnets 2-10

Instances Auto-scaling

Layer 4: Virtual Private Cloud

10.0.0.0/16

Groups Route tables

Requirement #1:

- CloudFront
- Global low latency

Requirement #2 and #3:

- Multi-AZ Application load Balancer
- Auto Scaling (High availability)

Simple Explanation of layers:

Layer 1: CloudFront CDN

This is a global cache system that stores copies of the website at 600+ locations worldwide so users can get fast delivery from any server nearby.

Benefits of this: this will help users far from my primary data center get fast content delivery of my website. (speed)

Layer 2: Application Load Balancer (ALB)

This layer receives all incoming user requests and directs them to whichever server is currently available and healthy. It helps distribute traffic evenly across multiple servers and performs health checks (verifies servers are working). Lastly ALB provides single entry point.

Layer 3: Auto Scaling Group (ASG)

Basically, ASG is a self-managing system that automatically creates new servers when traffic is high and removes servers when traffic is low. ASG handles traffic spikes without manual intervention. It also saves money by removing unused instances while ensuring website stays available during high demand.

Layer 4: VPC (Virtual Private Cloud)

This is the private, isolated network in AWS where you control everything-what servers exist, how they connect, and who can access what. This helps to keep security of resources (resources are isolated from others) and control who can access what. Configuring network exactly how I need it.

VPC: uses CIDR block 10.0.0.0/16, divided into public and private subnets across 2 availability zones. My VPC also includes Public Subnets and Private Subnets that support ALB to publicly accessible so users can reach my website and provide safety to ALB to access them via security groups respectively.

Security Groups: these are firewall rules that control traffic in and out.

i.e. ALB security group allows HTTP (port 80) from anywhere, allows all outbound

EC2 security group allows HTTP (port 80) only from ALB, allows all outbound

- Availability zones (Multi-AZ) , CloudWatch Monitoring and IAM Roles

Availability zones (multi-AZ) are basically multiple geographic locations within the same region for redundancy. This benefits us when AZ-1 fails, AZ-2 keeps serving users and provides high availability (achieves 99.99%) uptime SLA.

CloudWatch Monitoring: this is AWS monitoring service that watches my infrastructure metrics. It is helpful because it enables auto scaling (responds to CPU metrics), provides visibility (helps see how the infrastructure is performing) and sends alerts (notifies when something is wrong).

IAM Roles: these are Permissions that allow EC2 instances to interact with AWS services. Why it helps: Security, servers have minimal required permissions; allows instances to send metrics to CloudWatch.

VPC FOUNDATION – NETWORKING LAYER

This is a snippet of my VPC, where it spans 2 AZs with subnets. Public subnets host ALB and Private subnets host EC2. Security groups enforce least-privilege access.

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
  enable_dns_hostnames = true
}
resource "aws_subnet" "public_az1" {
  vpc_id = aws_vpc.main.id
  cidr_block = "10.0.1.0/24"
  availability_zone = data.aws_availability_zones.available.names[0]
}
resource "aws_subnet" "private_az1" {
  vpc_id = aws_vpc.main.id
  cidr_block = "10.0.11.0/24"
  availability_zone = data.aws_availability_zones.available.names[0]
}
# Repeat for AZ-2 (10.0.2.0/24 public, 10.0.12.0/24 private)
```

ALB REQUEST DISTRIBUTION LAYER

ALB in public subnets distributes traffic. Health checks every 30 seconds marks instances healthy/unhealthy. Only healthy instances receive traffic from CloudFront.

Snippet:

```
resource "aws_lb" "main" {
  name = "iu-alb-v2"
  load_balancer_type = "application"
  subnets = [aws_subnet.public_az1.id, aws_subnet.public_az2.id]
}
resource "aws_lb_target_group" "main" {
  name = "iu-tg-v2"
  protocol = "HTTP"
  port = 80
  vpc_id = aws_vpc.main.id
  health_check {
    interval = 30
    healthy_threshold = 2
    path = "/"
    matcher = "200"
  }
}
```

EC2 AND AUTO SCALING: COMPUTE LAYER

Launch template configures instances: Ubuntu 24.04 + Nginx, ASG maintains 2-10 instances across private subnets. CloudWatch monitors CPU: >70% scale up, <30% scale down.

```
resource "aws_launch_template" "main" {
  name_prefix = "iu-lt-"
  image_id = data.aws_ssm_parameter.ubuntu_ami.value
  instance_type = "t3.micro"
  user_data = base64encode(<<-EOF
  #!/bin/bash
  apt-get update && apt-get install -y nginx
  systemctl start nginx
  # Custom HTML with instance metadata
  EOF
  )
}
```

```
resource "aws_autoscaling_group" "main" {
  min_size = 2
  max_size = 10
  desired_capacity = 2
  launch_template { id = aws_launch_template.main.id }
}
```

CLOUDFRONT (CDN) GLOBAL DISTRIBUTION

CloudFront sits in front of ALB, caching content at 600+ global edge locations. Fulfills conception requirement#1 “Global Low Latency”. Supports all HTTP methods. TTL: 1-24 hours for dynamic updates.

```
resource "aws_cloudfront_distribution" "main" {
  enabled = true
  comment = "IU Cloud Programming CDN"
  price_class = "PriceClass_100" # US/Europe/Asia
```

```
origin {
  domain_name = aws_lb.main.dns_name
  origin_id = "ALB"
  custom_origin_config {
    origin_protocol_policy = "http-only"
  }
}
default_cache_behavior {
  allowed_methods = ["GET", "HEAD", "OPTIONS", "PUT", "POST",
"PATCH", "DELETE"]
  cached_methods = ["GET", "HEAD"]
  target_origin_id = "ALB"
  forwarded_values {
    query_string = false
    cookies { forward = "none" }
  }
  default_ttl = 3600
  max_ttl = 86400
}
```

COMPLETE TRAFFIC FLOW: 4 LAYER ARCHITECTURE

Internet Users → **CloudFront** → **ALB** → **ASG** → **EC2 instances**

1. User requests from anywhere globally
2. CloudFront intercepts at 600+ edge locations
3. If cached, CloudFront serves immediately (low latency)
4. If not cached, CloudFront requests from ALB origin
5. ALB performs health checks on target group

6. ALB routes only to healthy instances in private subnets
7. Instances respond with custom HTML (instance ID, hostname)
8. CloudFront caches response for 1-24 hours (based on TTL)
9. ASG monitors CPU: scales 2-10 instances automatically
10. Multi-AZ redundancy ensures 99.99% availability

TERRAFORM ORGANIZATION – 7 FILES, 600+ LINES

- **Main.tf**

VPC, subnets, route tables, security groups | 17 resources

- **ALB.tf**

Load balancer, target group, listener | 3 resources

- **EC2.tf**

IAM roles, launch template instance profile | 5 resources

- **ASG.tf**

Auto Scaling Group, policies, CloudWatch alarms | 4 resources

- **CloudFront.tf**

Cloudfront distribution, origin config, cache behavior | 1 resource

- **Variables.tf**

6 input variables with default values

- **Outputs.tf**

15+ outputs including cloudfront domain and ALB DNS

FULFILLED REQUIREMENTS FROM CONCEPTION PHASE:

Requirement #1: Global Low Latency

Conception: CloudFront CDN | Development: Implemented cloudfront.tf
600+ edge locations worldwide | Caching at edge | <50ms latency globally

Requirement #2: High Availability

Conception: Multi-AZ + ALB + ASG | Development: Fully implemented
2 AZs for redundancy | ALB health checks | 99.99% SLA

Requirement #3: Auto-Scaling

Conception: CPU-based scaling | Development: Fully implemented
ASG 2-10 instances | CloudWatch CPU monitoring | 70% scale up / 30% scale down

GITHUB REPOSITORY:

GitHub: <https://github.com/CODEWITHNDAHIRO/DLBSEPCP01-cloud-programming-AWS>

Slide

